

2.4 Example: Quiz

In this more extensive example, we assume that you want to program a simple guessing game. The program selects a few questions at random from a pool of questions. It asks the questions and provides several multiple-choice answers marked with the letters a, b, c, and so on to choose from. The player enters a character string (e.g., ab if the first and fourth answers are correct), and the program analyzes the answer and asks the next question.

2.4.1 Class Design

This section is primarily concerned with organizing the classes required for the program. You're still inexperienced when it comes to class design, so you ask an AI tool for help. In the first, very detailed prompt, you outline the basic functions of the program and name the programming language.

You immediately make it clear to the AI tool that there are multiple-choice questions where *multiple* correct answers are permitted. This is important information for the design of the program. Without this information, ChatGPT will most likely assume that there is only one correct answer per question. If you develop the program under this assumption at this point, extensive changes will be necessary later to extend the functionality. It's much better to clarify this detail now.

Prompt: I would like to write a small quiz game in Java. There is a predefined pool of questions. When the program starts, the player has to answer five questions randomly chosen from the pool.

For each question, there is a set of possible answers (min. 2, max. 6). At least one answer has to be correct. Some questions may have more than one correct answer. In this case, ALL correct answers must be selected.

Please help me design the necessary classes. Don't provide any finished code yet. Just help me decide on good class names and define the right instance variables. Leave constructors and methods for later.

Designing a quiz game in Java requires clear and well-structured classes to manage questions, answers, and the gameplay itself. Below is a proposed class design with suggested instance variables:

- **QuizGame:** Manages the overall game flow and logic. Instance variables:
 - `List<Question> questionPool`: Holds all possible questions for the game.
 - `List<Question> selectedQuestions`: Stores the randomly chosen questions for the current game.
 - `int score`: Tracks the player's score.
 - `int currentQuestionIndex`: Keeps track of the current question in the game.
- **Question:** Represents a single quiz question. Instance variables:
 - `String text`: The text of the question.
 - `List<Answer> answers`: A list of possible answers for this question.
 - `List<Answer> correctAnswers`: A list of correct answers.
 - `int id`: Unique identifier for the question (optional, for easier management).
- **Answer:** Represents a possible answer to a quiz question. Instance variables:
 - `String text`: The text of the answer.
 - `boolean isCorrect`: Indicates whether the answer is correct.
 - `int id`: Unique identifier for the answer (optional, for easier management).
- **Player:** Represents the player in the game [...].
- **QuizUtil:** Utility class for helper methods (e.g., random selection, validation ...)

[detailed description]

The suggestion is definitely useful. You decide to initially organize the code into three classes: `Main`, `Question`, and `Answer`. You start the implementation

independently, set up the start code in `Main`, and program the trivial `Answer` class.

2.4.2 Implementing the Question Class

In the next prompt, you request support with programming the `Question` class. It's important that you tell your AI tool what code already exists because it has to reference it in the `Question` class.

Prompt: OK. I created a `Main` class to run/test the program. I also wrote the `Answer` class like this:

```
public class Answer {
    public String text;
    boolean isCorrect;

    // constructor
    public Answer(String text, boolean isCorrect) {
        this.text = text;
        this.isCorrect = isCorrect;
    }
}
```

Next, I want to implement `Question`. Please suggest initial code (only instance variables and the constructor, without the optional `id` variable).

```
import ...;

public class Question {
    public String text;
    public List<Answer> answers;
    public List<Answer> correctAnswers;

    // Constructor
    public Question(String text,
                    List<Answer> answers,
                    List<Answer> correctAnswers) {
        this.text = text;
        this.answers = answers;
        this.correctAnswers = correctAnswers;
    }
}
```

```
}  
}
```

2.4.3 A Pool of Questions for Testing

Before you look at the other methods of these classes, you need test material, that is, some quiz questions. Creating them is a rather dreary task that can be delegated to ChatGPT and others.

Prompt: As a base for testing the code, please provide a method for the `Main` class to initialize a pool of 10 geographic questions.

ChatGPT initially only provides questions with exactly *one* correct answer. The AI tool only adds questions with multiple matching answers on request.

Prompt: Perfect. However, you could include a few questions with more than one correct answer (e.g., rivers in Brazil)?

2.4.4 Methods for the Question Class

Now that we have the first quiz questions, it's now time to design the `Question` class. At least two methods are required here: one for displaying a question together with its solution and for entering the answer, and the second for evaluating the answer. In the following prompt, you ask for the implementation of the first method, specifying relatively precisely how the question should be presented (the answers should be marked with lower-case letters) and in which format the program expects the answer. To prevent the AI tool from outputting the entire class code, we ask that it only returns the new method.

Prompt: Add a method to the `Question` class to print the question text and all of the enumerated answers such as a), b), c), and so on. The method prompts the user for an answer string, for example, "ad" if answers a) and d) are correct. Provide code only for the method, not for the entire class.

The AI tool suggests two new methods: `askQuestion` implements the prompt's specifications and returns the answer as a character string. The

unsolicited `validateAnswer` method evaluates this character string and checks whether it contains all applicable answers, but no further letters for nonapplicable answers.

In this example, the `validateAnswer` method is the trickiest. But before you could even think about how to program the evaluation of the character string yourself, ChatGPT has already presented a ready-made method. This is annoying.

The code created up to this point can be found in the sample files for this book. At this point, we leave this example. Of course, various methods are still missing to complete the program (game sequence, random selection of five questions, presentation of the questions, display of the result [e.g., three out of five questions answered correctly], etc.). Maybe you want to complete the example yourself—that would be a good exercise in the use of AI tools. Or, can we motivate you with this example to implement this or a similar game yourself with the help of AI? You decide to what extent you want to be supported.

Overzealous Learning Aid

Unfortunately, ChatGPT and others tend to be overzealous when it comes to their use as a learning aid. The AI tools often provide more code than necessary (even if you explicitly request the opposite in the prompt). As soon as the code appears on the screen and you've read it, you're already influenced.

Of course, nobody is forcing you to adopt the code, but the temptation is great. The learning effect would be greater if you developed more methods yourself. If you do fail with a problem, you can always ask for improvement tips for your half-finished code.

2.4.5 UML Diagram

The graphical summary in the form of a Unified Modeling Language (UML) diagram of classes, variables or attributes, or properties and methods is a valuable aid to maintaining an overview.

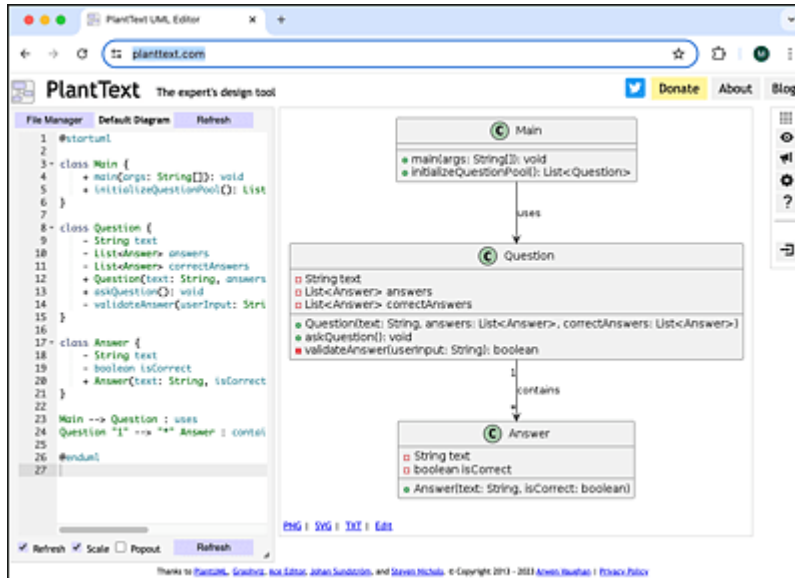


Figure 2.1 UML Diagram of the Quiz Program in a PlantUML Representation

As of the time of writing (early 2025), the current AI tools are unfortunately still hopelessly overwhelmed in this respect. As the following prompt shows, not even the ASCII representation of the diagram is free of errors. (Note the right-hand edge of the boxes!)

Prompt: Can you present a UML diagram of the classes so far?

Certainly! Below is a UML diagram that represents the classes and their relationships as described so far for the quiz game. The diagram includes the Main, Question, and Answer classes with their respective attributes and methods.

```
+-----+
|           Main           |
+-----+
| + main(args: String[]): void |
| + initializeQuestionPool(): List<Question> |
+-----+
```

```

+-----+
|           Question           |
+-----+
| - text: String               |
| - answers: List<Answer>      |
| - correctAnswers: List<Answer> |
+-----+
| + Question(text: String, answers: List<Answer>,
|           correctAnswers: List<Answer>) |
| + askQuestion(): void        |
| - validateAnswer(userInput: String): boolean |
+-----+

```

You can avoid this restriction by asking the AI tool in the prompt to provide code for a common diagram tool instead of the diagram. Mermaid (<https://mermaid.live>) and PlantUML (www.planttext.com) are well suited for this purpose. There are websites for these libraries where you can insert the UML code and display it immediately.

Prompt: Please generate PlantUML code instead.

```

@startuml
class Main {
    + main(args: String[]): void
    + initializeQuestionPool(): List<Question>
}
...

```

2.4.6 AI Bot or Code Wizard?

The more extensive your code becomes and the more work you delegate to AI tools, the less suitable it is to use a chat system such as ChatGPT or a similar tool. The problem is that the AI tool loses track of the code if you develop parts of it yourself.

Of course, you can insert the current status of the program into the prompt from time to time or make it available via drag and drop. You can avoid this increasingly tedious back and forth as the number of classes increases by using a code wizard that has permanent access to the entire code.

An argument against the use of a code wizard is that it not only becomes active when explicitly requested, but also makes suggestions for additions every time code is entered or changed. This is useful for writing code efficiently, but less so for learning.